

Student name: Ting Li; Duo Xu

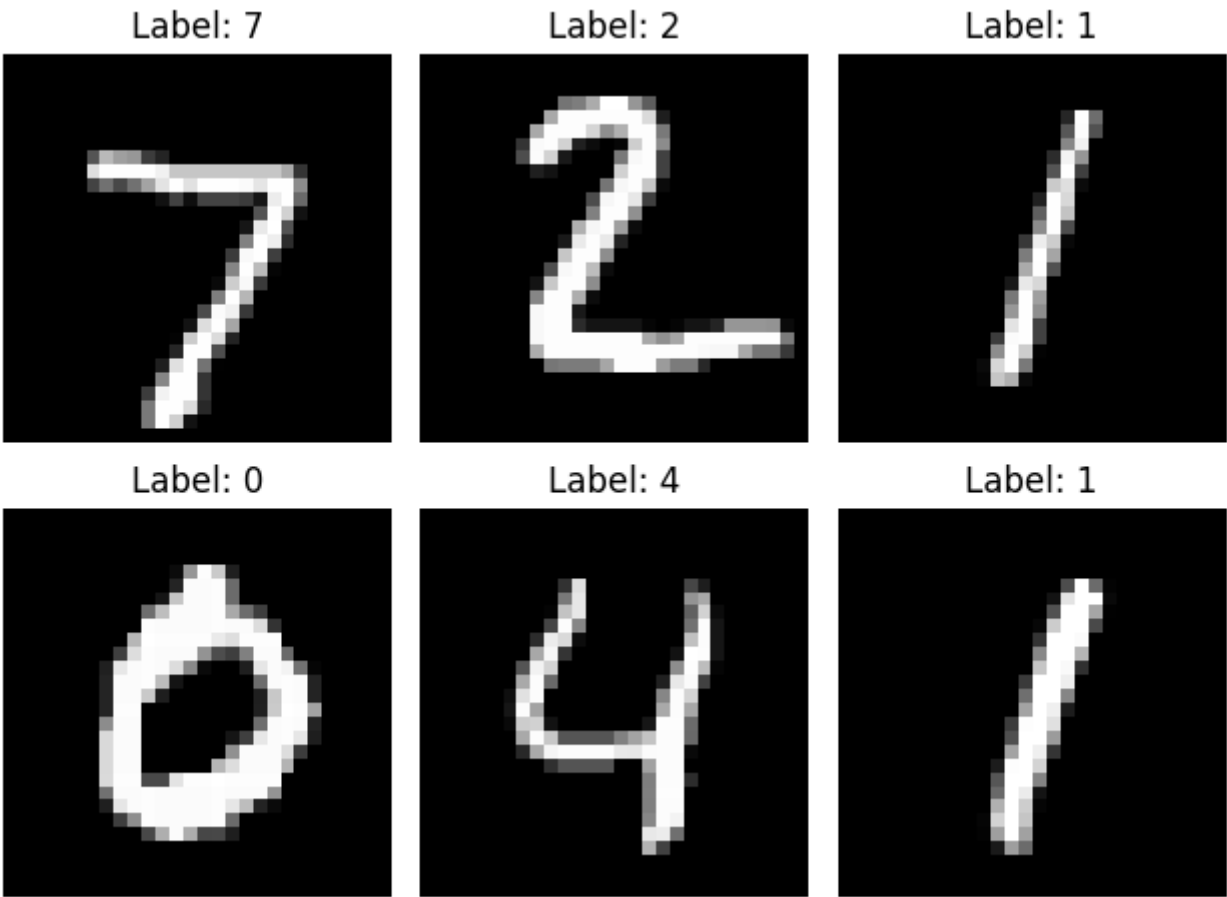
Travel day usage: 2 totally, and 1 day per person

Description of Project:

This project explores how deep learning models recognize digits and fashion items using the MNIST and Fashion-MNIST datasets. In **Task 1**, we built and trained a simple CNN to recognize hand-written digits, tested it on both standard and custom digits, and analyzed its predictions. In **Task 2**, we visualized what the network learns in the early layers. In **Task 3**, we applied transfer learning by fine-tuning the MNIST model to classify three Greek letter (alpha, beta, gamma), using both small(~9 per letter) and big training datasets(~70 per letter). In **Task 4**, we tested how changing CNN structure, like the number of filters, dense nodes, and dropout rate affects model performance. We trained 64 combinations automatically and analyzed accuracy, training time, and loss. Finally, in the **Extension**, we replaced the first CNN layer with fixed Gabor filters.

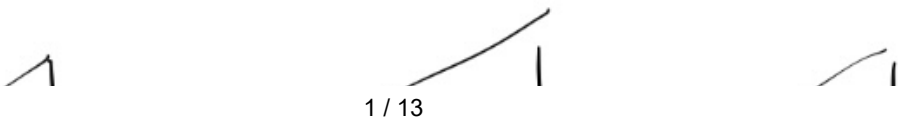
Task 1.

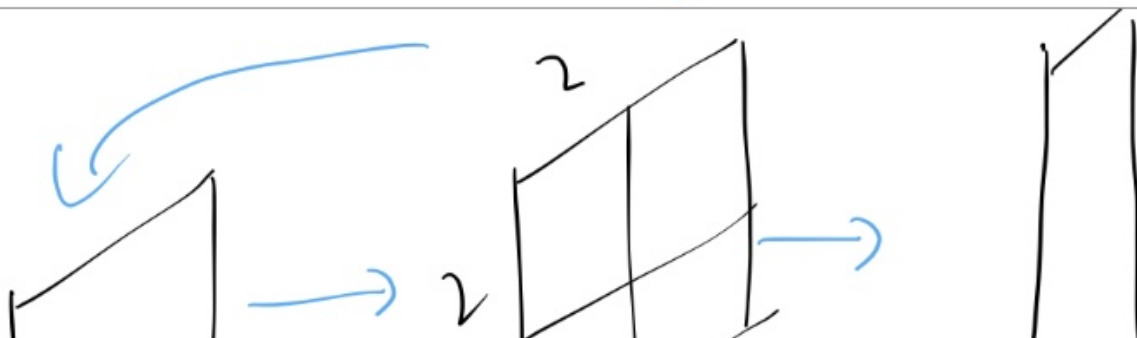
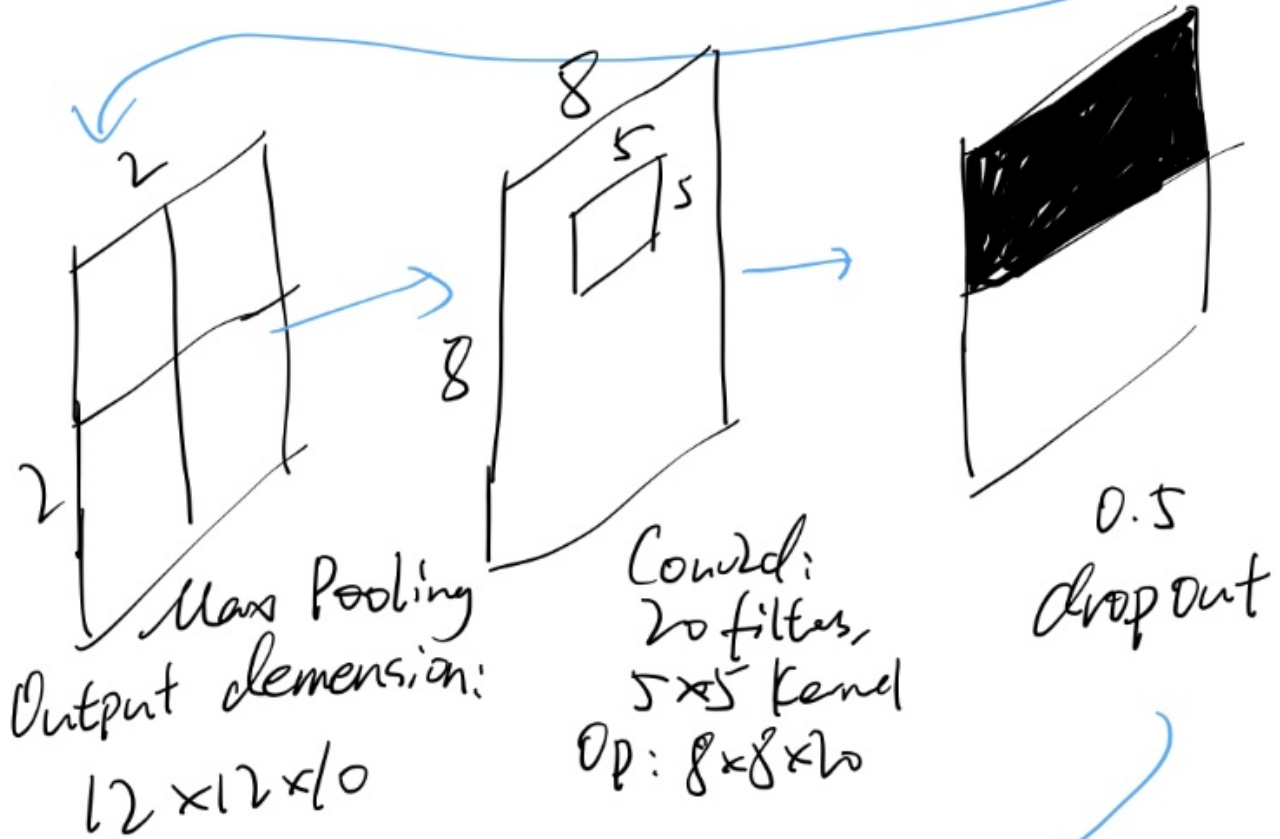
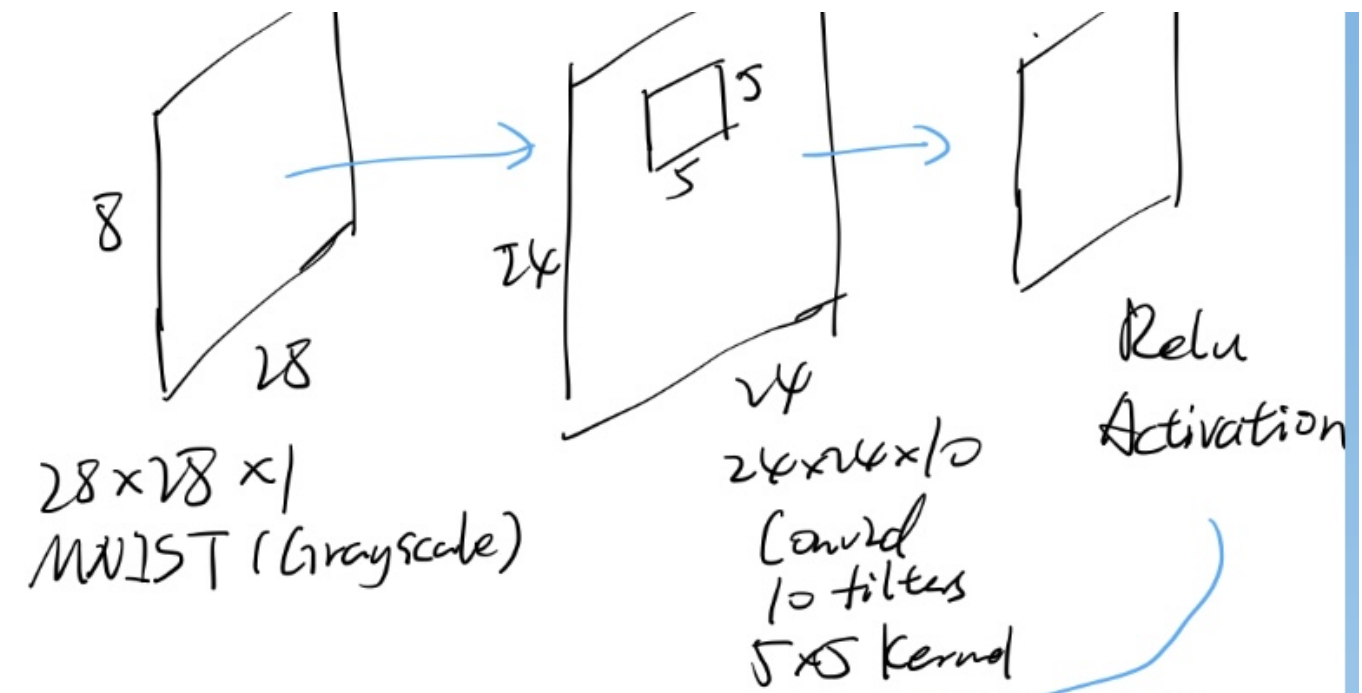
A.

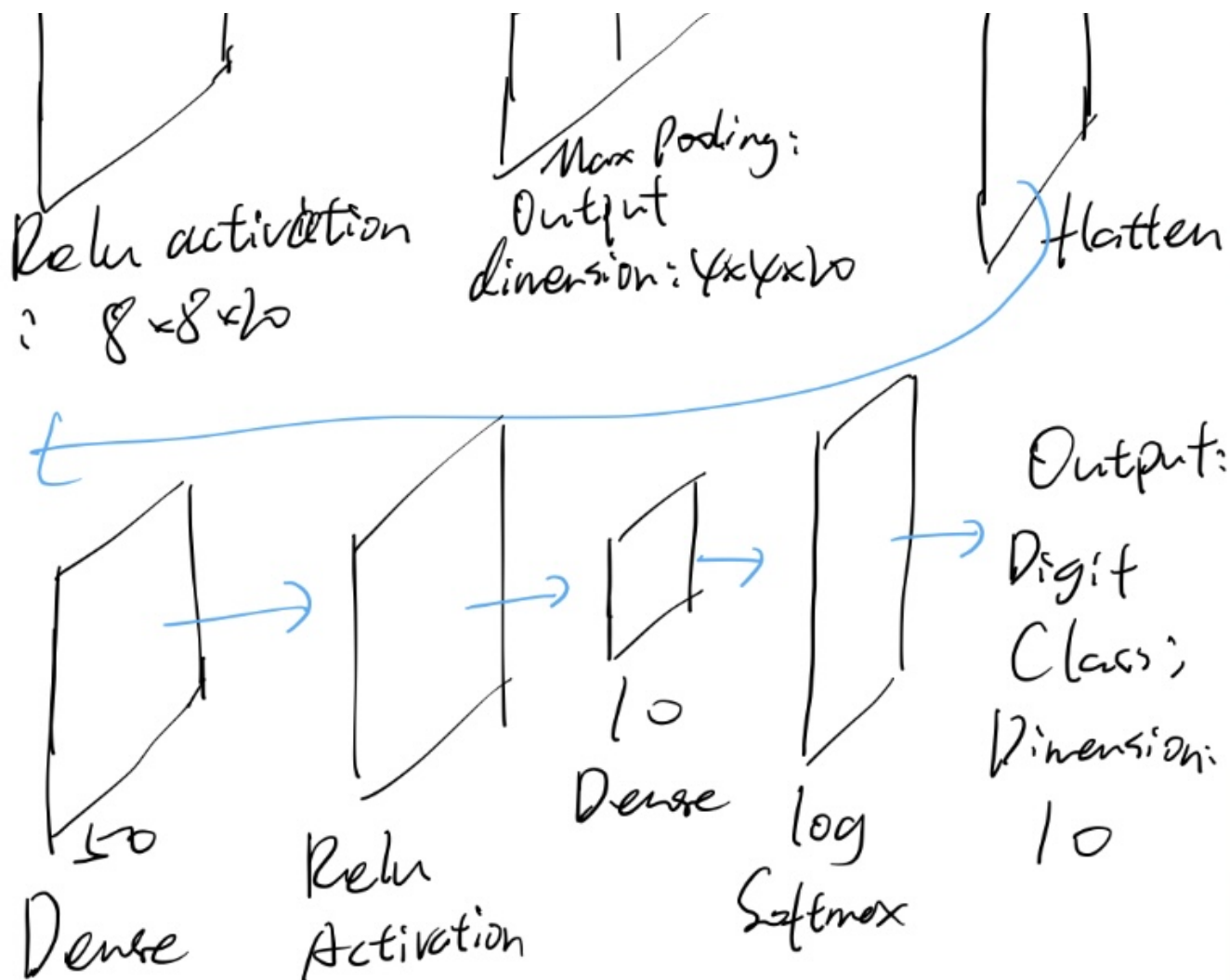


Shows the first six MNIST test images used to evaluate the trained digit classifier.

B.

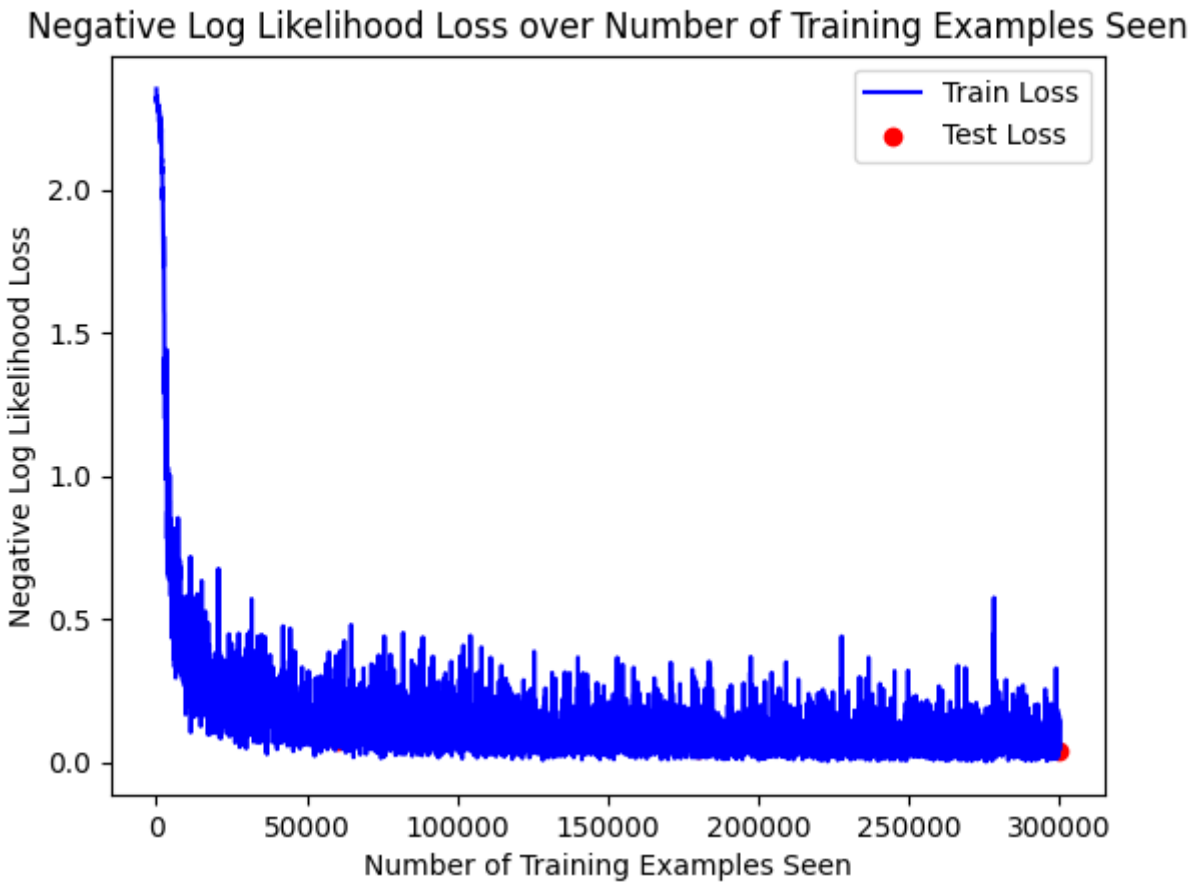






Visualizing the architecture of our convolutional neural network used for MNIST digit recognition.

C.

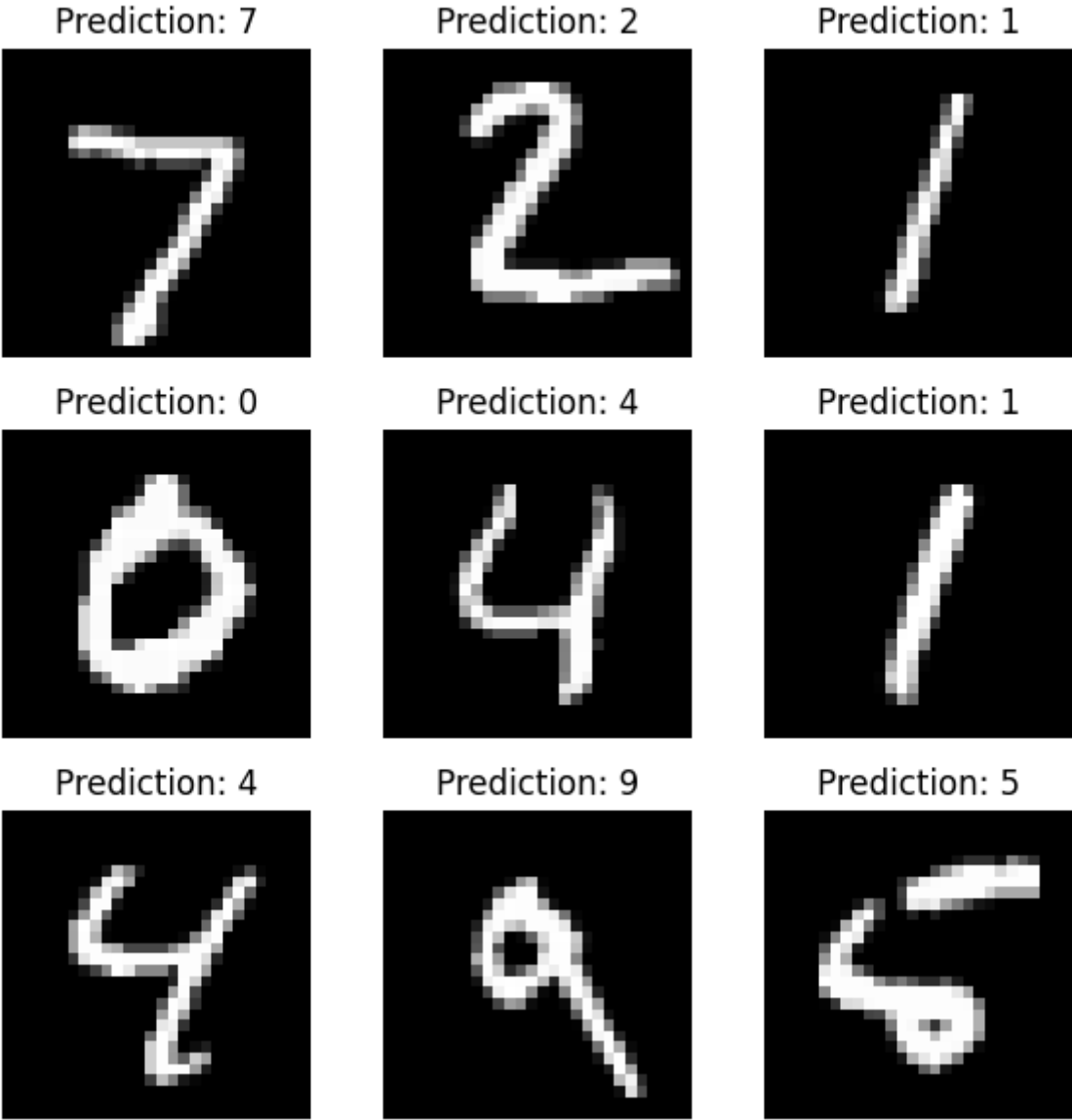


Plots

training loss over five epochs, showing how the model learns over time.

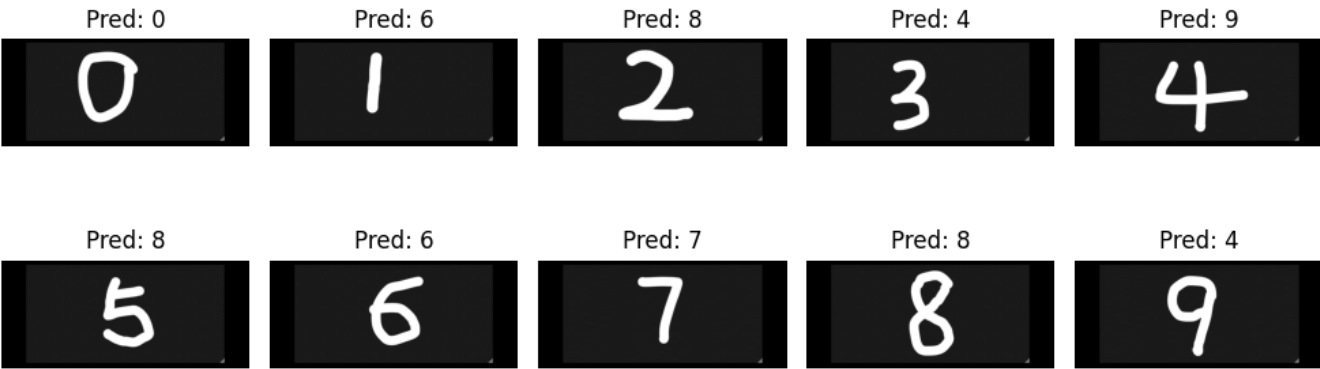
E.

```
[Running] python -u "d:\personalAshley\NEUstudy\25Spring\5330pattern\HW\PJ5\task1_evaluate.py"
Image 0: Output: [-21.13, -18.16, -14.04, -16.65, -18.85, -21.33, -33.47, -0.00, -21.27, -16.03], Predicted: 7, True: 7
Image 1: Output: [-15.29, -13.05, -0.00, -24.17, -25.31, -26.61, -17.28, -22.92, -20.66, -23.16], Predicted: 2, True: 2
Image 2: Output: [-16.45, -0.00, -10.32, -15.83, -10.67, -13.89, -12.96, -8.69, -11.73, -13.39], Predicted: 1, True: 1
Image 3: Output: [-0.00, -23.73, -13.43, -15.36, -17.08, -12.45, -7.30, -18.07, -12.79, -11.27], Predicted: 0, True: 0
Image 4: Output: [-19.32, -19.02, -13.67, -14.80, -0.00, -17.37, -17.26, -13.85, -12.15, -8.20], Predicted: 4, True: 4
Image 5: Output: [-18.40, -0.00, -12.11, -19.69, -12.15, -17.48, -17.15, -9.61, -14.43, -15.38], Predicted: 1, True: 1
Image 6: Output: [-30.93, -18.05, -23.05, -21.90, -0.00, -21.62, -25.67, -14.77, -10.57, -11.90], Predicted: 4, True: 4
Image 7: Output: [-18.59, -18.14, -13.38, -12.46, -4.81, -10.75, -22.48, -12.42, -10.22, -0.01], Predicted: 9, True: 9
Image 8: Output: [-11.38, -15.45, -14.29, -11.17, -12.75, -0.24, -2.15, -15.41, -2.32, -6.11], Predicted: 5, True: 5
Image 9: Output: [-28.45, -25.04, -25.45, -19.59, -6.16, -19.43, -33.26, -11.90, -12.97, -0.00], Predicted: 9, True: 9
```



Demonstrates predictions for ten random MNIST test digits with their actual values.

F.

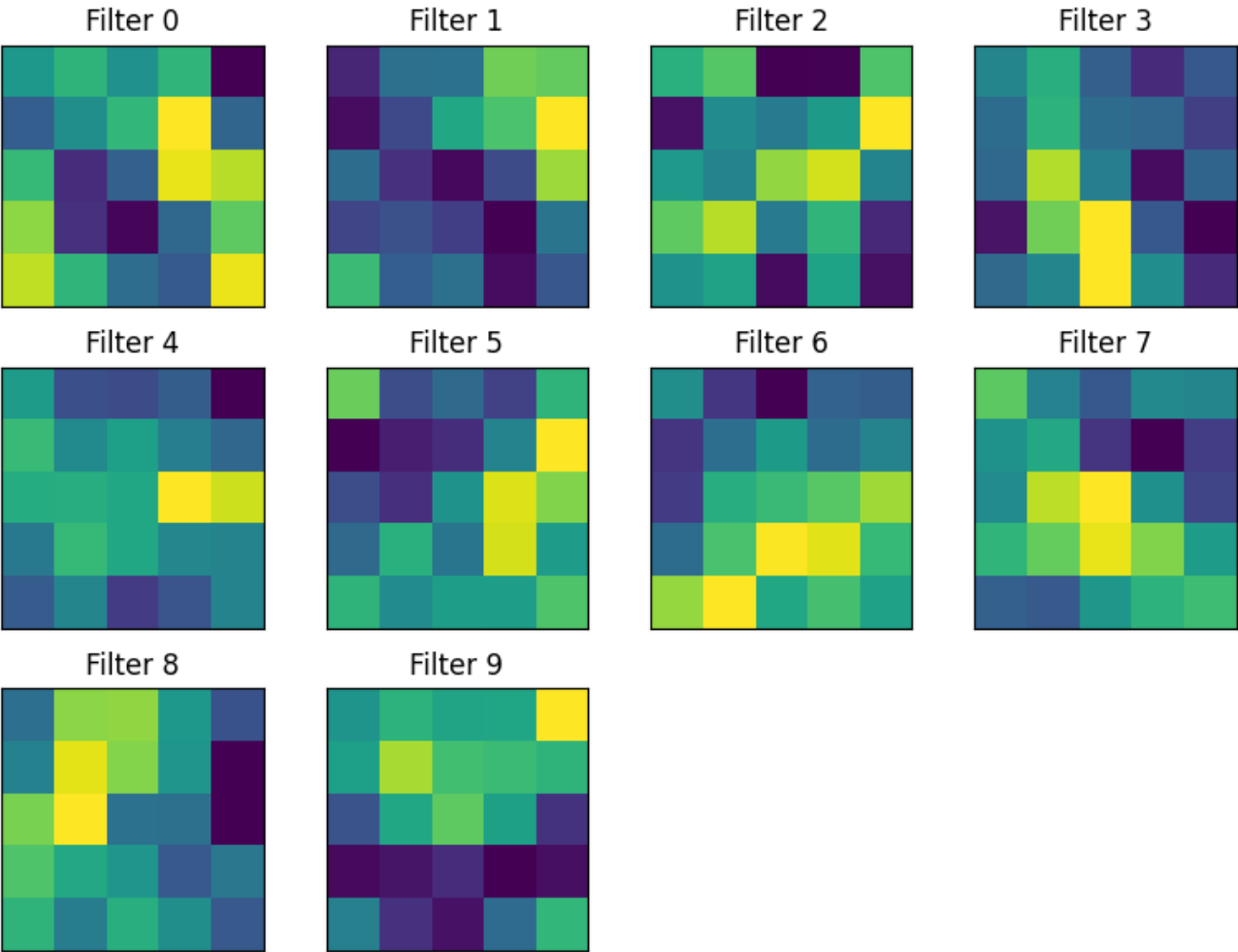


From above, we can tell that hand-written digit 1, 2, 3, 4, 5 are not correctly predicted, while the rest digits are correctly predicted. We think the reason might be those letters not fit in MNIST digits' standard.

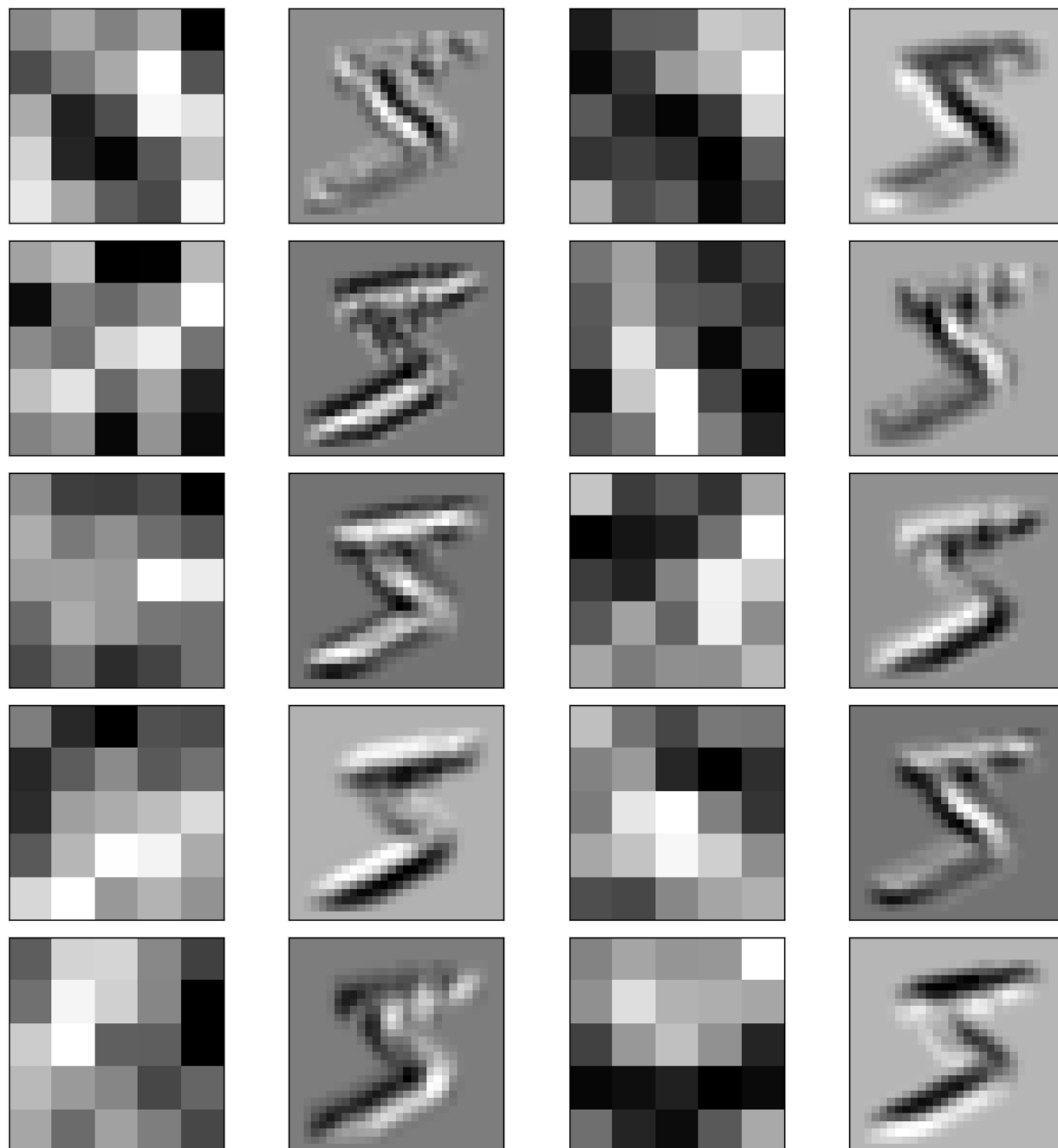
Task 2.

A.

We think it looks like Sobel (edge detect filter) since we can see high contrast in one direction.



B.



We think the result makes sense. The filters learned by the network respond to parts of the digit that match their shape, like edges or curves.

Task 3.

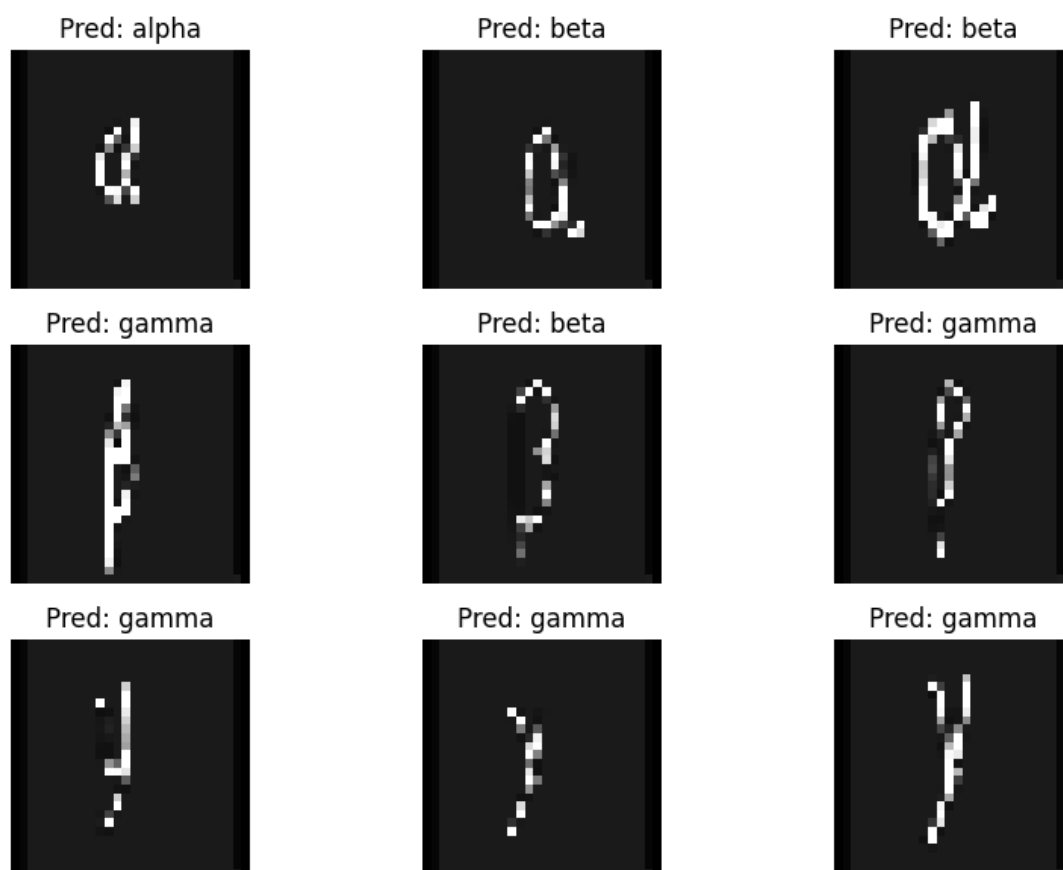
Train result of greek_train file:



```
[Running] python -u "d:\personalAshley\NEUstudy\25Spring\5330pattern\HW\PJ5\task3_transfer_greek.py"
Epoch 1, Loss: 0.9810
Epoch 2, Loss: 0.6617
Epoch 3, Loss: 0.4434
Epoch 4, Loss: 0.3274
Epoch 5, Loss: 0.3130
Epoch 6, Loss: 0.2242
Epoch 7, Loss: 0.2078
Epoch 8, Loss: 0.2478
Epoch 9, Loss: 0.1644
Epoch 10, Loss: 0.1341
Epoch 11, Loss: 0.1186
Epoch 12, Loss: 0.1245
Epoch 13, Loss: 0.1370
Epoch 14, Loss: 0.1548
Epoch 15, Loss: 0.1214
Epoch 16, Loss: 0.1323
Epoch 17, Loss: 0.0855
Epoch 18, Loss: 0.0820
Epoch 19, Loss: 0.0947
Epoch 20, Loss: 0.0631
```

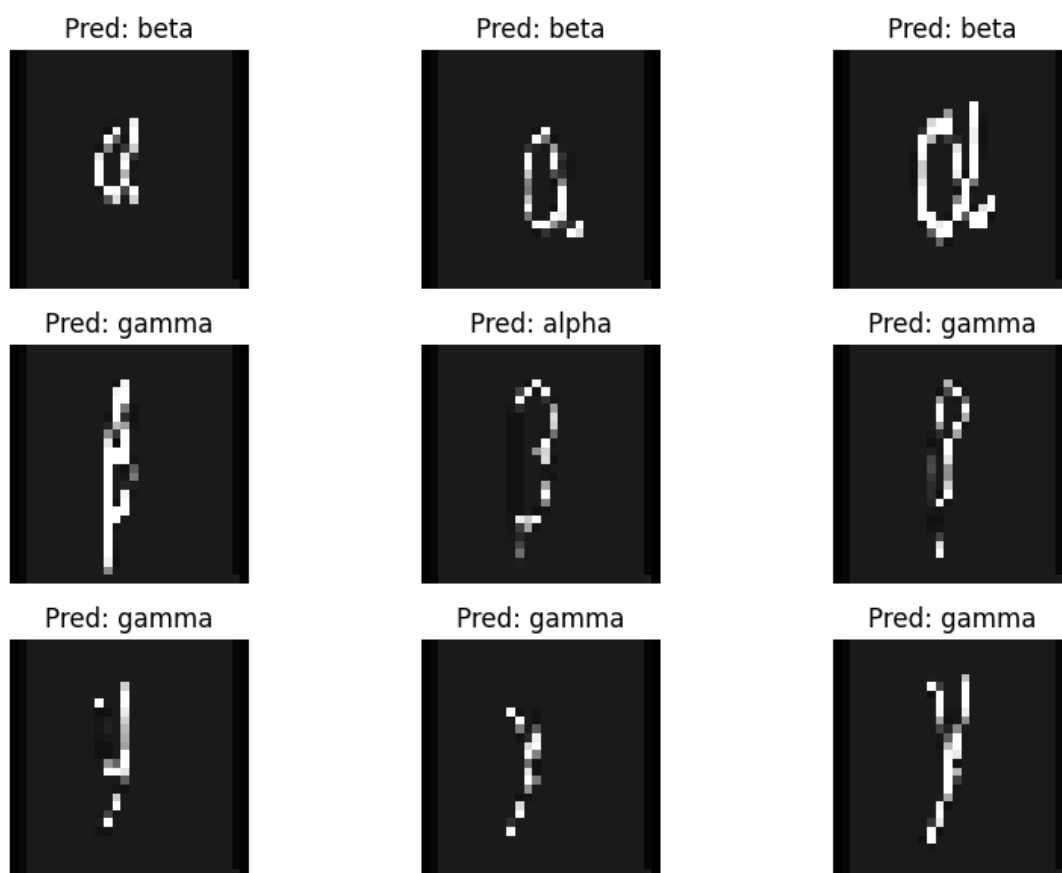
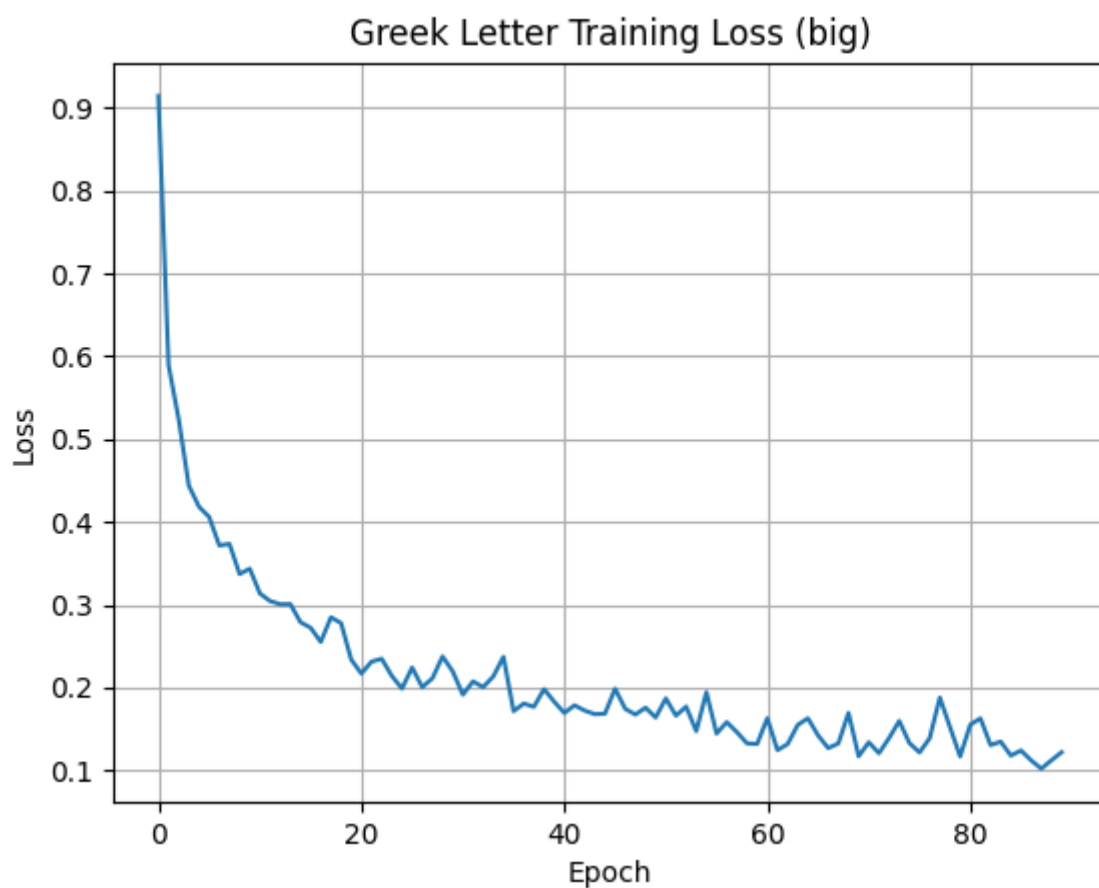
From above two picture, we think the number of epoch should be at least of more than 10 (loss hence < 0.15).

Test result of self-handwritten greek letters based on small and addition(big) images trained:



From above image, we can see 5 out of 9 letters could be correctly identified, which is one alpha, one beta and all three gamma.

Besides, we continued to train a model still for these three letters using about 70 images per letter from addition folder's images. These images were processed to match the MNIST digit style and used to fine-tune a pre-trained MNIST model. However, the final test results were not ideal, only the handwritten gamma letters were predicted correctly, while most alpha and beta letters were misclassified, given by the training loss is promising.



Please see the link of my hand written three greek letter: [one drive link](#)

Task 4.

A. Plan

We aim to find a good CNN structure for the Fashion-MNIST classification task by experimenting with different network settings. Specifically, we explore three dimensions of the CNN:

1. Number of convolution filters – how many filters are in the convolution layer.
2. Number of fully connected (dense) nodes – how big the fully connected layer is.
3. Dropout rate – how much dropout to apply during training.

We try the following values for each dimension:

- Convolution filters: [5, 10, 20, 30]
- FC nodes: [32, 64, 128, 256]
- Dropout rate: [0.3, 0.5, 0.7, 0.9]

This gives a total of 64 combinations ($4 \times 4 \times 4$), and we evaluate all of them automatically using Python. For each model, we record:

- Test accuracy
- Training time
- Average training loss

We use a linear search approach: for each setting, we hold two parameters constant and vary the third, so we can observe the effect of each change independently.

B. Predictions

To our best knowledge, we hypothesize that the best model should be: 30 filters, with 128 FC nodes, and dropout rate 0.5.

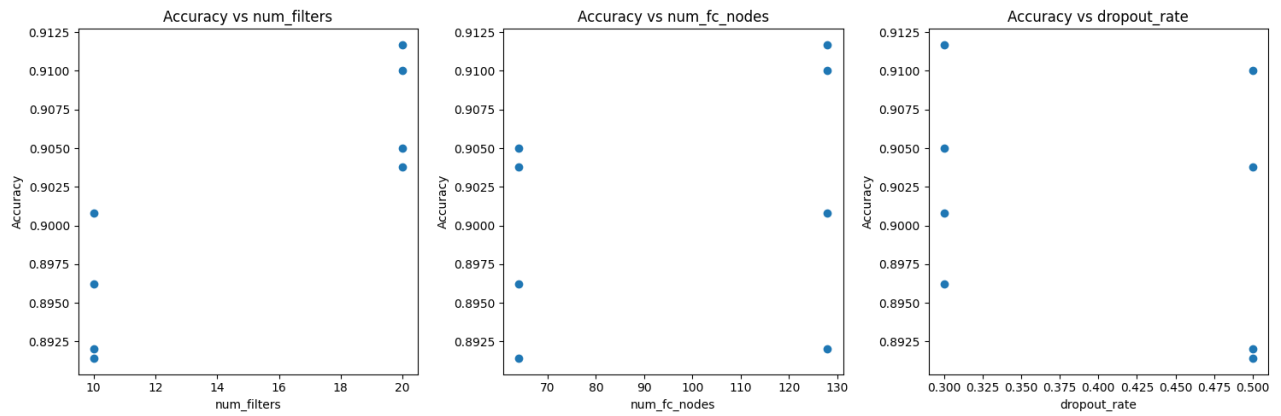
C. Plan execution and Hypotheses Evaluation

The best model (see result CSV image below) is the one with 20 convolution filters, 128 FC nodes, and dropout rate 0.3, because it achieves the highest accuracy and lowest loss among all tested configurations. While its training time is a bit longer (around 151s), it's still acceptable compared to others (e.g., 147–150s).

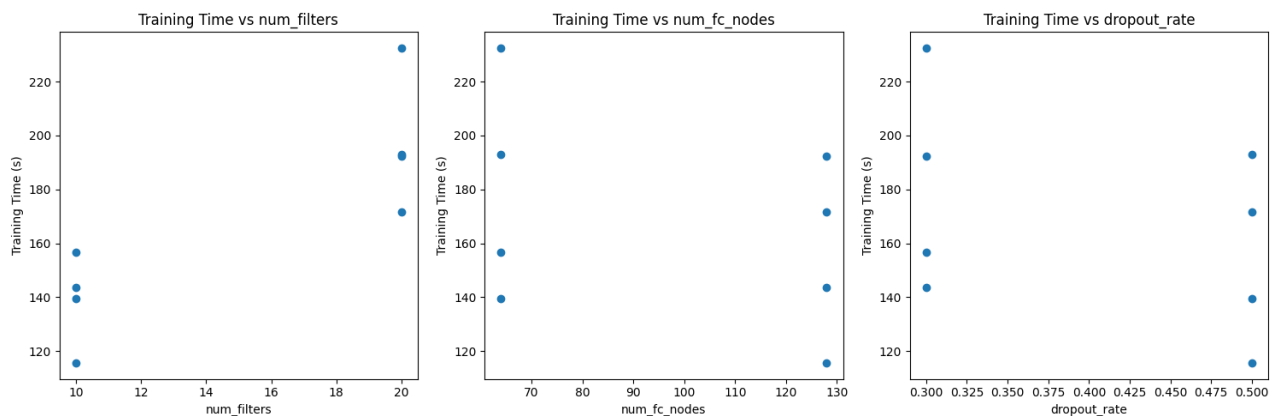
num_filters	num_fc_nodes	dropout_rate	accuracy	training_time	loss
10	64	0.3	0.8966	101.14328598976135	0.36598730014839653
20	64	0.3	0.9049	145.99143767356873	0.3271927000124698
10	64	0.5	0.8905	101.95913052558899	0.40550779583834135
20	64	0.5	0.9056	147.66449785232544	0.34783836610313423
10	128	0.3	0.9005	104.6896619796753	0.34714617684070487
20	128	0.3	0.9063	151.72379088401794	0.3096908401793191
10	128	0.5	0.896	103.22398924827576	0.3938414334234144
20	128	0.5	0.9082	150.40893173217773	0.34359312841632983

Below are related plots.

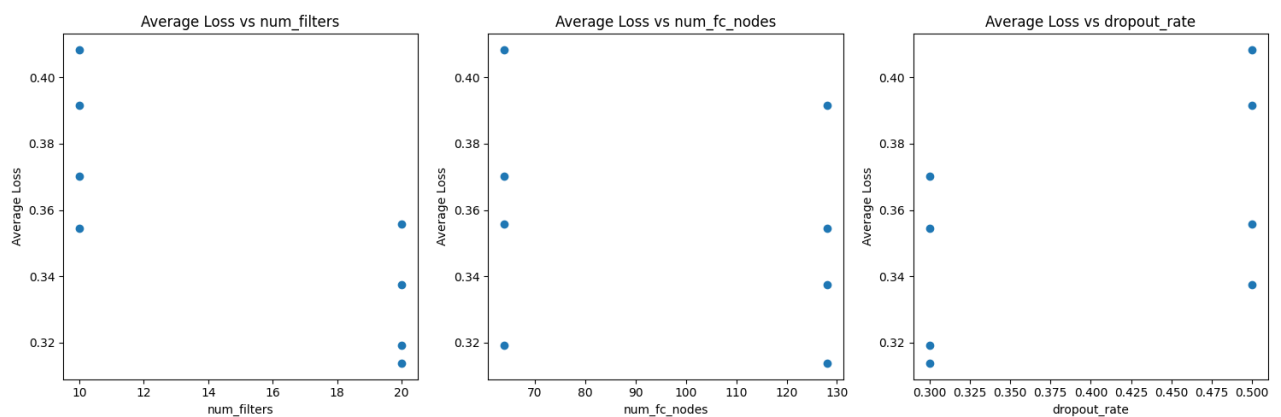
- Test accuracy VS three dimensions:



- Training time VS three dimensions:



- Average training loss VS three dimensions:



Extension.

Replacing the first convolutional layer with fixed Gabor filters resulted in a test accuracy of **86.81%** (see image below), which is reasonably close to the fully trainable CNN baseline (**~91–92%**)[1].

This demonstrates that Gabor filters offer a strong starting point for visual recognition tasks, especially in environments with limited data or limited time since we noticed the run time is significantly decreased.

However, trainable filters is still better.

```
PS D:\personalAshley\NEUstudy\25Spring\5330pattern\HW\PJ5> python task_extension_gabor.py
Training Gabor-based model...
Test Accuracy using Gabor filters: 0.8681
```

Reflection:

This project gave us a deeper understanding of how convolutional neural networks (CNNs) work. We learned how to build a model, train it, and improve its performance by adjusting layers, filters, and dropout rates. It was interesting to see how small changes in the network could affect both accuracy and training time. Testing with handwritten greek letters showed us the challenges of applying a model to different data, even it looks similar. We also learned how to automate experiments and analyze results using plots. Replacing the first layer with Gabor filters was a new idea for us, and we saw that fixed filters can still perform quite well. Overall, we gained useful experience in model design, training, testing, and evaluation. This project helped build both our technical skills and confidence in working with deep learning models.

Reference

[1] Zalando Research — Fashion-MNIST Dataset. <https://github.com/zalando-research/fashion-mnist>